Ejercicio HOF - Top productos por facturación neta 2024

Abre el fichero **Top Productos.txt** que tienes en la carpeta de Ejercicios. Verás que contiene un Array de pedidos y uno de clientes vetados. Realiza un programa JS que realice los siguientes **pasos**:

1. Filtra los pedidos del año 2024, con el estado "enviado", y que no pertenezcan a clientes vetados.
2. Obtén de cada pedido las líneas. Para cada línea calcula importe neto = unidades × precio × (1 − dto).
3. Agrega por SKU acumulando ingresos netos y unidades vendidas.
4. Devuelve el TOP 3 por ingresos netos, con estructura:
5. Muestra la colección resultante de mayor a menor ingresos, y sin mutar los arrays originales.

Ejercicio HOF - Ranking académico

Abre el fichero **Ranking Académico.txt** que tienes en la carpeta de Ejercicios. Verás que contiene un Array de alumnos. Realiza un programa JS que siga las siguientes **reglas**:

1. Trabajamos solo con alumnos activos.
2. Para cada módulo, usa la mejor calificación entre nota y recuperación (si existe).
3. Si mH es true, suma 0.25 a la nota final de ese módulo (tope 10).
4. La media del alumno es la media aritmética de todos sus módulos tras ajustes.
5. No mutar alumnos ni los arrays internos que uses.

Teniendo en cuenta estas reglas, **devuelve**:

1. Un ranking descendente de { nombre, media }, con dos decimales.
2. Una distribución por tramos con el número de alumnos en cada categoría:

{"[0,5)"], "[5,7)", "[7,9)", "[9,10]"}

Resultado esperado (en esta forma):

```
{
  ranking: [ { nombre, media }, ... ],
  distribucion: { "[0,5)": n, "[5,7)": n, "[7,9)": n, "[9,10]": n }
}
```

Ejercicio HOF – Índice de calidad

Abre el fichero **Índice Calidad.txt** que tienes en la carpeta de Ejercicios. Verás que contiene un Array de posts y otro de bloqueados. **Reglas** para el score de cada post:

1. Ignora comentarios de usuarios bloqueados.
2. Ignora comentarios con reports ≥ 2.
3. Pondera los likes según rol: user = 1, pro = 1.5 (puedes aplicar otras ponderaciones si justificas).
4. Score del post = suma de likes ponderados de sus comentarios válidos.
5. Devuelve una lista de posts con { id, titulo, score } ordenada de mayor a menor score. En empates, ordena alfabéticamente por título.
6. No mutar posts ni sus arrays internos.

Ejercicio HOF – Planificación semanal y horas lectivas

Abre el fichero **Planificación Semanal.txt** que tienes en la carpeta de Ejercicios. Verás que contiene un Array de horarios. **Tareas** para el programa:

1. Calcula el número total de horas lectivas por módulo a la semana. Ten en cuenta que algunos tramos duran 1h y otros más.
2. Calcula, para cada grupo, cuántas horas totales tiene a la semana.

Resultado esperado:

```
{
  horasPorModulo: [
    { modulo: "Cliente", horas: number },
    { modulo: "Servidor", horas: number },
    { modulo: "Interfaces",  horas: number },
    …
  ],
  horasPorGrupo: [
    { grupo: "DAW", horas: number },
    { grupo: "DAM-A", horas: number },
    …
  ]
}
```

Ejercicio HOF – Logs del Servidor y detección de IPs problemáticas

Abre el fichero **Logs de Servidor.txt** que tienes en la carpeta de Ejercicios. Verás que contiene un Array de logs.
Tareas para el programa:

1. Considera que una IP es "sospechosa" si tiene 3 o más intentos de login (endpoint /login) con estado 401.
2. Calcula, para cada IP:
    a. Número total de peticiones.
    b. Número de errores 5xx.
    c. Tiempo medio de respuesta (media de tiempoMs).

Resultado esperado:

```
{
  ipsSospechosas: [ "10.0.0.X", ... ],
  resumenPorIp: [
    {
      ip: "10.0.0.1",
      peticiones: number,
      errores5xx: number,
      tiempoMedio: number
    },
    ...
  ]
}
```

La lista resumenPorIp debe estar ordenada de mayor a menor por número de peticiones, y, en caso de empate, por IP ascendente.

Ejercicio HOF – Carrito con promociones

Abre el fichero **Carrito con Promociones.txt** que tienes en la carpeta de Ejercicios. Verás que contiene un Array de productos, otro de promociones, y un par de variables. **Tareas** para el programa:

1. Calcula el importe bruto de cada línea: unidades × precio.
2. Aplica las siguientes reglas de descuentos (sin mutar carrito):
    a. Promos de tipo "porcentaje" se aplican sobre productos de esa categoría.
    b. Promos de tipo "descuento_fijo" se aplican sobre el sku concreto.

3. Calcula el importe neto total del carrito tras aplicar todas las promociones de producto (sin incluir IVA ni envío todavía).
4. Aplica la promo "envio_gratis":
    a. Si el importe neto es mayor o igual al minimo → gastos de envío = 0
    b. Si no, gastos de envío = gastosEnvioBase.

5. Calcula:
    a. baseImponible = importe neto de productos.
    b. importeIVA = baseImponible × iva.
    c. total = baseImponible + importeIVA + gastosEnvio.

6. Devuelve un objeto final:

```
{
  lineas: [
    {
      sku,
      nombre,
      unidades,
      precioUnitario,
      descuentoAplicado,// total descuento por esa línea
      subtotalNeto       // tras descuento
    },
    ...
  ],
  resumen: {
    baseImponible,
    iva: importeIVA,
    gastosEnvio,
    total
  }
}
```